Homework08

Task 1 (2 points):

Create a new database on Neon.

1. On the Dashboard page of your Neon account, click on the button labeled **"Connect"** towards the top right of the screen.

2. Click the drop-down menu under **“Database”** then click **“Create new database”**.

3. Under **“Database name”** type “mystery” then click **“Create”**.

4. Ensure “mystery” now appears under **“Database”**.

5. At the bottom of the window, click **“Show password”** then **“Copy snippet”**.

6. You’ve just copied the connection string.

Task 2 (2 points):

Open your GitHub repository in Codespace.

1. In the terminal run the command: `pip install -r requirements.txt`

2. In the terminal run the command: `cd HW/Homework08/Mystery/`

3. In the terminal run the command: `cp .env.example .env`

4. In the **.env** file, paste the copied connection string from task 1 after: **DB_CONNECTION_STRING=**

5. You’ll now need to parse the connection string to get the **DB_HOST** and **DB_PASSWORD** values.

   Example:

   ```
   postgresql://neondb_owner:password@ep-example-123456-pooler.us-east-1.aws.neon.tech/mystery?sslmode=require&channel_binding=require
   ```

6. The **host** can be found between **@** and the first **/** after **@**. The **password** can be found between the first **:** before **@** and **@** in the connection string.

   ```
   DB_HOST=ep-example-123456-pooler.us-east-1.aws.neon.tech
   DB_PASSWORD=password
   ```

7. Run **test.py** to ensure a successful connection to the mystery database.

Task 3 (2 points):

Create a file called **Database.py** inside **HW/Homework08/Mystery/** and at the top of the file, import the following:

```
psycopg2
os
csv
tabulate from tabulate
load_dotenv from dotenv
```

Create a class named **Database** with the following methods:

1. `def __init__(self, ddl_file: str ="ddl.sql", data_dir="DATA") -> None:`
   Define the following variables:
   self.connection assigned `None`
   self.ddl_file assigned `ddl_file`
   self.data_dir assigned `data_dir`
   self.relations assigned `[]`

2. `def connect(self) -> None:`
   Connect to the database using the **connect_to_db()** function from setup.py as a guide. Instead of returning the connection/None, assign it to self.connection.

3. `def close(self) -> None:`
   If the self.connection is not None, close self.connection and assign it None. Then assign an empty list to self.relations.

4. `def __enter__(self) -> "Database":`
   Call self.connect() and return self.

5. `def __exit__(self, exc_type, exc_val, exc_tb) -> None:`
   Call self.close().

6. `def get_relations(self) -> list[str]:`
   Define this method using the get_relation_names() function from load.py. Use self.connection and self.ddl_file.

7. `def create_tables(self) -> None:`
   Define this method using the create_tables() function from setup.py. Use self.connection and self.ddl_file. At the end of the try block, assign self.relations the return of self.get_relations().

8. `def load_data(self) -> None:`
   Define this method using the load_data() function from load.py. Use self.connection and self.data_dir.

9. `def setup(self) -> None:`
   Call self.create_tables() and self.load_data().

10. `def execute_query(self, query) -> None:`
    Define this method using the execute_query() function from query.py. Use self.connection.

11. `def show_tables(self) -> None:`
    Define this method using the show_tables() function from load.py. Use self.connection. (Loop through self.get_relations())

12. `def drop_tables(self) -> None:`
    Define this method using the drop_tables() function from drop.py. Use self.connection. At the end of the try block, assign an empty list to self.relations.

13. `def version(self) -> None:`
    Define this method using the version() function from test.py. Use self.connection.

Task 4 (2 points):

Create file called **main.py** and import Database from Database as DB.

1. Instantiate a DB object called **db**. Then connect to the database calling the connect() method.

2. Create the tables by calling **db.setup()**.

3. Create a list called **cleanup_queries** that contains 4 queries. Each query should be encased in triple quotes.(i.e. `"""UPDATE table SET attribute = NULL WHERE condition;"""`)
   Queries should be:
   a. Update the person table such that license_id equals NULL if the license_id is not in the drivers_license table.
   b. Update the person table such that ssn equals NULL if the ssn is not in the income table.
   c. Alter the person table adding license_id as a foreign key that references the drivers_license table.
   d. Alter the person table adding ssn as a foreign key that references the income table.

4. Loop through **cleanup_queries** calling **db**'s **execute_query()** method for each query in the list.

5. Create a list called **queries** that contains all queries used to solve the sql murder mystery of lab02. Each query should be encased in triple quotes.(`"""SELECT * FROM table WHERE condition;"""`)

6. Loop through the **queries** list calling **db**'s **execute_query()** method for each query in the list.

7. Close the DB connection.

8. Run **main.py**.

Push your work to GitHub using the following commands:

```
git pull
git add .
git commit –m "homework08 upload"
git push
```